

AMR Final Project

Julia Töws, Rene Hadder, Tim Dörpinghaus

Abstract—In this project, we brought three navigation functionalities from the course Autonomous Mobile Robots to the Robile platform: path and motion planning with A* and a potential field planner, Monte Carlo localisation with our own particle filter, and fringe-based exploration together with gmapping for SLAM. All functionalities work reliably in the simulation: the robot reaches its goals, localises itself globally in a known map and explores an unknown environment completely. On the real robot, we had to solve several additional problems, such as different clock times of the laptop and the robot, invalid laser measurements and collisions with the corners of the robot. A narrow passage in our test arena is still difficult for the robot, if it enters the passage at an angle.

Index Terms—Mobile robot navigation, Monte Carlo localisation, frontier-based exploration

I. PROJECT DESCRIPTION

A. Motivation

During the course Autonomous Mobile Robots, we implemented several basic navigation methods, but mostly in simulation. The motivation of this project is to show what we have learned in the course and to bring these methods to a real robot. This gives us experience with a realistic project, where the robot, its sensors and the environment do not behave as ideally as in the simulation.

B. Problem Statement

The task is to deploy three functionalities from the course on the Robile platform, a four-wheeled omnidirectional robot with a laser scanner at its front. The three parts build on each other:

- 1) **Path and motion planning:** The potential field planner from the assignments has to be ported to the real robot. It is combined with a global path planner (e.g. A*), which finds a rough path of waypoints on a map. The potential field planner then moves the robot from waypoint to waypoint and avoids obstacles.
- 2) **Localisation:** We have to implement our own particle filter (Monte Carlo localisation) that estimates the pose of the robot in an existing map.
- 3) **Environment exploration:** The robot has to explore an unknown environment on its own. An exploration component selects the poses to explore, ideally at the map fringe (the boundary between the explored and the unexplored region), and a SLAM component creates the map.

All functionalities first had to work in the simulation and then on the real robot.

*Submitted to the Department of Computer Science at Hochschule Bonn-Rhein-Sieg in partial fulfilment of the requirements for the degree of Master of Science in Autonomous Systems

‡Submitted in September 2026

C. Proposed Approach

We implemented all parts as nodes of one ROS 2 (Humble) package in Python (Fig. 1):

- **A* path planner:** A* searches the shortest path on the occupancy grid map. The obstacles are grown by the robot radius (configuration space), so that the path keeps a distance to walls. The path is sampled into waypoints, which are sent to the navigator one after the other.
- **Potential field navigator:** The waypoint attracts the robot, and the closest obstacle measured by the laser scanner repels it. The sum of both forces gives the driving direction and speed.
- **Particle filter:** Many particles represent possible poses of the robot. They are moved with the odometry (motion model), weighted by comparing the laser scan with the map (measurement model) and resampled according to their weights.
- **Fringe-based explorer:** The explorer selects the closest reachable cell at the map fringe as the next goal. The map itself is created by *slam_gmapping*, an existing SLAM implementation based on a particle filter.

Placeholder: overview of the nodes and topics (explorer → /goal → path_planner → /waypoint → potential_field_navigator → /cmd_vel; particle_filter or slam_gmapping → map → odom)

Fig. 1: Overview of the implemented nodes and the topics between them.

II. DESCRIBING THE CODE

This section describes the implemented nodes and the functionalities we added to the basic algorithms. The code and a more detailed description of all topics and parameters are available in the README of our repository¹.

A. Task 1: Path and Motion Planning

Path planner. The node *path_planner* receives a goal and the occupancy grid map. A* searches the shortest path from the robot to the goal, using the direct and diagonal neighbours of a cell and the Euclidean distance as heuristic. Occupied cells are grown by 0.35 m, so that the path keeps a distance to obstacles.

¹<https://github.com/t1md0e/amr-team-jrt>

If no path is found, e.g. in a narrow passage, this distance is reduced step by step, so that the path still stays in the middle of the passage. The path is sampled into waypoints, and only one waypoint at a time is sent to the navigator. Intermediate waypoints only have to be passed within 0.3 m, while the goal has to be reached within 0.1 m.

Potential field navigator. The node *potential_field_navigator* moves the robot to the current waypoint. The attractive force $\mathbf{F}_{att}$ is a unit vector towards the waypoint. The repulsive force is calculated from the clearance d between the robot and the closest obstacle:

$$\mathbf{F}_{rep} = -\eta \left(\frac{1}{d} - \frac{1}{\rho_0} \right) \frac{1}{d^2} \mathbf{e}_{obs} \quad \text{for } d < \rho_0, \quad (1)$$

with $\eta = 0.1$, $\rho_0 = 0.6$ m and the unit vector $\mathbf{e}_{obs}$ towards the obstacle. The robot turns towards the sum of both forces and slows down when it turns, when it is close to obstacles and when it is close to the waypoint. If the desired direction is to the side or behind the robot, it rotates in place first.

Compared to the basic potential field planner from the assignments, we added:

- **Rectangular footprint:** d is the clearance between the rectangular footprint of the robot ($0.76 \text{ m} \times 0.47 \text{ m}$) and the obstacle, so that also the corners keep a distance. Only the closest obstacle is used, because summing over all laser points counts a wall many times and creates local minima.
- **Safety:** Below a clearance of 0.1 m, the robot only moves away from the obstacle. Before a rotation in place, the rotation is simulated with the footprint, and the robot rotates the other way round if the short way is blocked. If both ways are blocked, it moves back along its driven path.
- **Narrow passages:** If obstacles are close but the footprint can move in a straight line to the waypoint, the robot moves there without rotating, as it is omnidirectional.
- **Obstacle memory:** The laser scanner only looks to the front. Therefore, laser points are remembered for some time, so that obstacles next to and behind the robot are still known.

B. Task 2: Localization

The node *particle_filter* implements Monte Carlo localisation with 2000 particles. When the map is received, the particles are spread uniformly over all free cells (global localisation). Alternatively, a start pose can be set in RViz, and the particles are sampled around it. The filter is only updated after the robot has moved or rotated a little (0.05 m or 0.05 rad). Every update has three steps:

- 1) **Motion update:** Every particle is moved with the odometry motion model (rotation, translation, rotation), with Gaussian noise on every part.
- 2) **Measurement update:** For 30 laser beams, the expected range is calculated for every particle by casting a ray through the map. The weight of a particle is the likelihood of the measured ranges (Gaussian with $\sigma = 0.4$ m around the expected range, plus a small probability for random measurements).

- 3) **Resampling:** Particles are drawn proportionally to their weights.

The estimated pose is the weighted mean of the particles. The node publishes it and the transform from the map to the odometry frame, which the path planner and the navigator use.

We added the following extensions:

- **Recovery (augmented MCL):** If the short-term average of the particle weights drops below the long-term average, random particles are added. This helps if the robot is lost or kidnapped.
- **Symmetric environments:** The particles are not resampled before the robot has moved, as a single scan often fits several places of a symmetric map.
- **Partial maps:** Beams that end in an unknown cell of the map get a uniform likelihood, so that the filter also works with maps that are not completely explored.

C. Task 3: Environment Exploration

The map is created by *slam_gmapping*, a ROS 2 port of the gmapping SLAM package, which also publishes the pose of the robot in the map. Our node *explorer* selects the goals:

- 1) It finds the fringe cells, i.e. free cells next to unknown cells, and groups them into connected regions. Small regions are ignored.
- 2) Occupied cells are grown by 0.5 m, so that the robot fits at the goal and can rotate there.
- 3) A wavefront search (breadth-first search over the free cells) finds the closest reachable fringe cell that is at least 1 m away. This cell is sent to the path planner as the next goal.
- 4) The goal orientation points towards the unknown cells around the goal, so that the laser scanner looks into the unexplored region once the robot arrives.

A new goal is selected when the goal is reached, when the region around it has already been explored on the way, or when the robot makes no progress. Goals without progress are put on a blacklist.

We added the following functionalities:

- **Initial step:** At the start, the cell of the robot itself is still unknown, because the laser scanner is at the front. Therefore, the robot first moves a small step forward.
- **Driving direction:** If the goal was already explored on the way, the next goal is searched close to the old goal. Otherwise, the robot would turn around, e.g. inside a narrow passage, as soon as it sees into the room behind it.
- **Stopping and saving:** The exploration can be stopped after a maximum time or if no new area is explored for some time. The map is saved regularly, so that it can be used for the localisation of task 2.

III. EXECUTING THE CODE

We first tested all tasks in the simulation and then on the real robot. This section describes both environments and shows the results.

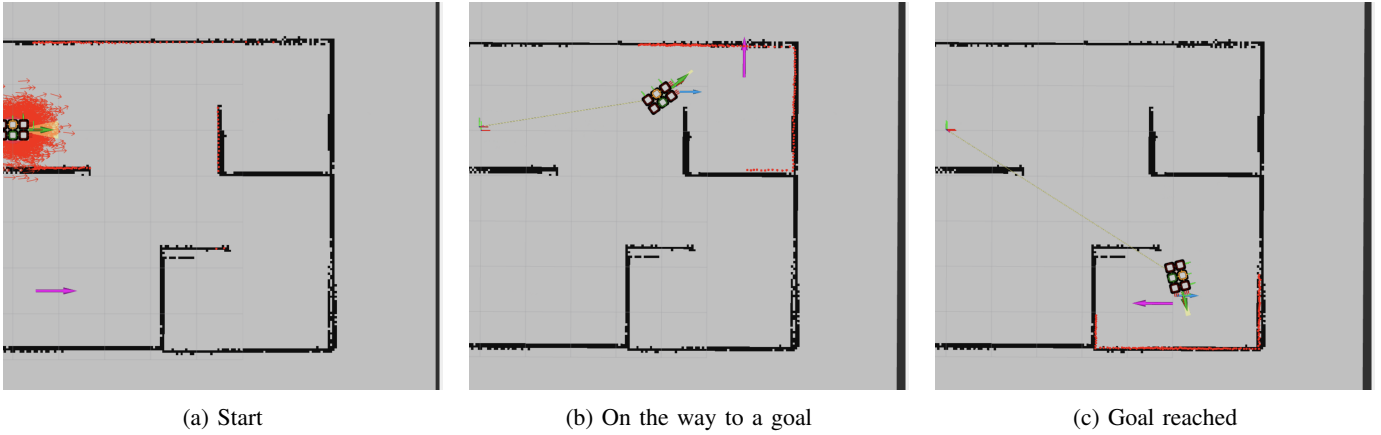


Fig. 2: Task 1 in the simulation (RViz): The robot follows the waypoints of the A* path to the goal (purple arrow).

A. The Simulation

Environment. We used the Robile simulation in Gazebo Classic with ROS 2 Humble. It ran in Docker containers: partly on an Ubuntu computer in a container with Ubuntu, and mostly on a Mac, where the container emulates the amd64 architecture, because Gazebo Classic is not available for ARM processors. As the Gazebo interface did not open in the container, we only ran the simulation server and used RViz to see the robot, the laser scan, the map and the particles. Most tests were done in the world *closed_walls* (about $7\text{ m} \times 7\text{ m}$) and some in a house world as a harder test with open doors.

Task 1: Path and motion planning. For this task, the robot was localised with our particle filter in a map created before. We sent three goals one after the other (Fig. 2). The robot reached all goals without collisions in about 30 s to 50 s each, with a final position error of about 0.02 m to 0.16 m. Video: https://github.com/t1md0e/amr-team-jrt/blob/main/documentation/videos/task1_navigation.mp4

Task 2: Localisation. We started the particle filter without a start pose, so the particles were spread over the whole map (Fig. 3). While the robot was moving, the particles converged to the correct pose after about 15 s to 90 s, depending on the start position. After that, the error of the estimated position was about 0.01 m to 0.15 m. In some runs, the filter first converged to a wrong but similar-looking place and corrected itself after 30 s to 80 s thanks to the random particles. Video: https://github.com/t1md0e/amr-team-jrt/blob/main/documentation/videos/task2_localisation.mp4

Task 3: Environment exploration. The robot started in an unknown environment and explored it on its own (Fig. 4). In the final runs in *closed_walls*, the complete map (about 52 m^2) was created in about 90 s to 120 s, and the exploration stopped by itself when no reachable fringe was left. Video: https://github.com/t1md0e/amr-team-jrt/blob/main/documentation/videos/task3_exploration.mp4

B. The Real Robot

Environment. In the lab, we used a Robile with four wheels and a laser scanner at its front. The ROS 2 nodes ran in a Docker container with Ubuntu on a laptop with Ubuntu,

which was connected to the robot via WLAN. We tested in the corridor of the university with boxes as obstacles (Fig. 5) and in an arena that we built ourselves: two rectangular areas connected by a narrow passage. Compared to the simulation, we reduced the speed of the robot to at most 0.3 m/s and 0.8 rad/s.

Task 1: Path and motion planning.

Task 2: Localisation.

Task 3: Environment exploration. In the arena, the robot explored the first area, but it did not pass the narrow passage to the second area (see Sec. IV-C).

IV. CHALLENGES

This section describes the biggest problems we faced during the project and how we solved them.

A. Simulation

Setup. We ran the simulation in Docker containers: some simulations on an Ubuntu computer in a container with Ubuntu and ROS 2, and most of them on a Mac. On the Mac, the container had to emulate the amd64 architecture, which caused several problems: the Gazebo interface did not open, RViz sometimes froze, and processes of old runs kept running and disturbed new runs. We therefore only used the Gazebo server with RViz and restarted the container before every test. In addition, the ROS 2 port of gmapping sometimes stopped after the first scan, so we restarted it automatically in this case.

Navigation. At first, the robot drove in circles around the waypoints or got stuck in front of walls. The reason was that the repulsive forces of all laser points were summed up, which made the repulsion of a wall 10 to 20 times stronger than the attraction. We solved this by using only the closest obstacle, by slowing down close to the waypoint and by rotating in place if the waypoint is to the side or behind the robot. Planning the path in the configuration space kept the path away from walls.

Localisation. In symmetric environments, the particle filter sometimes converged to a wrong place, because a single scan fits several places of the map. It helped to resample only after the robot had moved, to use more particles (2000 instead of

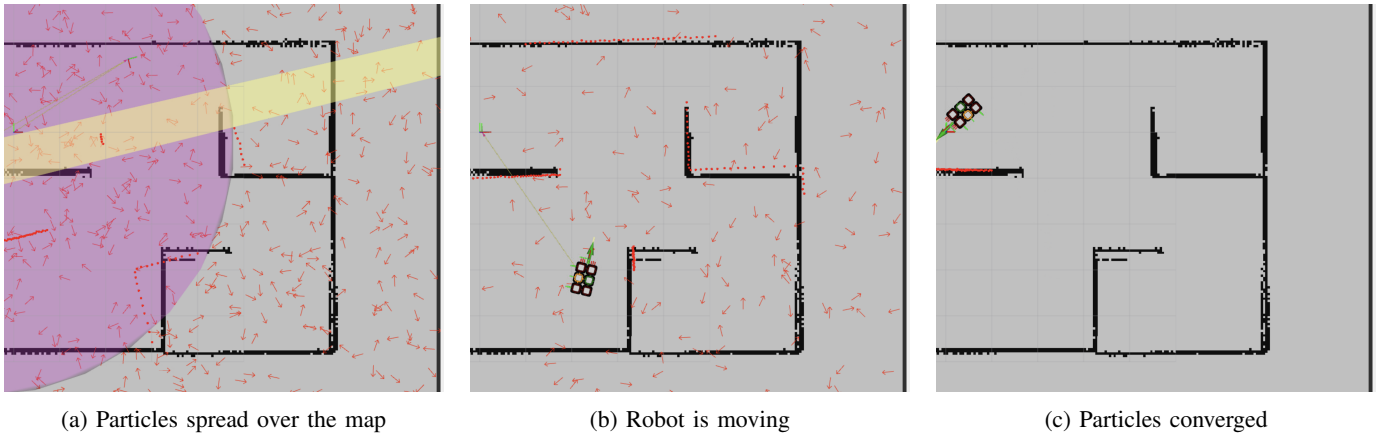


Fig. 3: Task 2 in the simulation: global localisation with the particle filter (red arrows are particles).

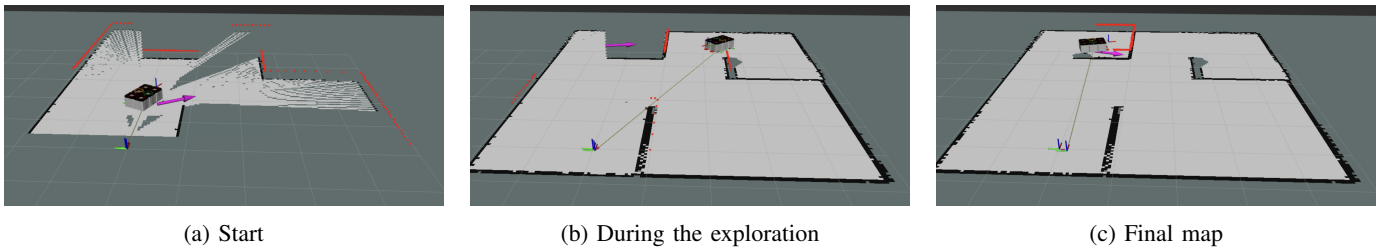


Fig. 4: Task 3 in the simulation: The map created by gmapping grows while the explorer selects goals at the map fringe.

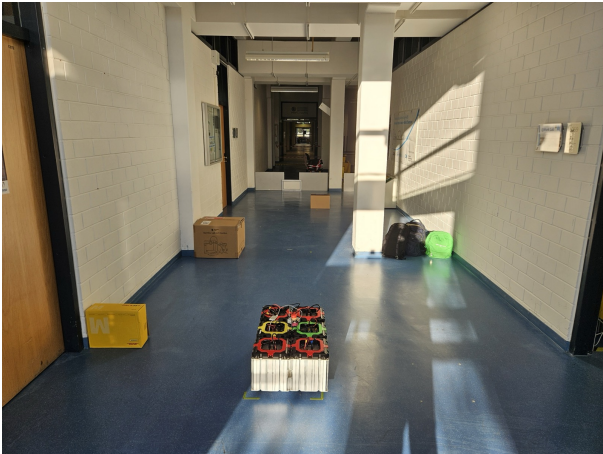


Fig. 5: The Robile in the corridor, with boxes as obstacles.

500) and a larger measurement noise ($\sigma = 0.4$ m instead of 0.2 m). Look-alike rooms are still a weak point, but the random particles usually correct a wrong estimate after some time.

Exploration. The explorer first chose goals very close to the robot, so the robot crept slowly along the fringe, and in the house world it left the building through an open door. We added a minimal goal distance of 1 m, an optional exploration boundary and stopping criteria. We also found that the explorer and the path planner converted coordinates to map cells differently, which caused many failed path searches.

Placeholder: photo of the arena with the narrow passage

Fig. 6: Our arena in the lab: two areas connected by a narrow passage.

Placeholder: RViz screenshot or photo of task 1 on the real robot

Fig. 7: Task 1 on the real robot.

B. Connecting to the Robot

Ubuntu laptop. With the Ubuntu laptop, the robot did not move at first. One reason was that our navigator expected the name of the laser frame from the simulation (*base_laser_front_link*), while it is called *base_laser* on the

Placeholder: RViz screenshot of the particles on the real robot

Fig. 8: Task 2 on the real robot.

Placeholder: map of the lab / arena created by the exploration

Fig. 9: Map created by the exploration on the real robot.

real robot. Now, the frame is taken from the scan messages. The second reason was that the clock of the laptop was about 59s ahead of the clock of the robot. Because of the different timestamps, the transforms between the frames could never be found, and the explorer did not send any goals. After we set the clock of the laptop back by this difference, the robot drove.

macOS and Docker. We also tried to control the robot from a Mac, where ROS 2 runs in a Docker container. Docker on macOS runs the containers in a virtual machine, which blocks the automatic discovery of ROS 2 (multicast). We configured the communication to send messages directly to the robot (unicast). The robot and the container found each other, but no topics arrived. The reason is that Docker on macOS drops network packets larger than about 1400 bytes, which is the case for most ROS 2 messages. A relay program that forwarded the packets did not solve the problem either, so we went back to the Ubuntu laptop.

C. From the Simulation to the Real Robot

Sensor data. The real laser scanner reports invalid measurements as 0, which created huge repulsive forces. These measurements are now ignored. The range of the real scanner (up to 60 m) is also much larger than in the simulation, so we limited the ray casting of the particle filter to 10 m.

Collisions with the corners. During a rotation close to a wall, the robot touched the wall with one of its corners, because the navigator treated the robot as a point. We added the rectangular footprint of the robot: the clearance is measured from the footprint, and a rotation is only started if it is free.

Narrow passage. In our arena, the robot did not pass the narrow passage (about 0.8m wide, estimated from the laser data). It turned left and right in front of it, moved back and tried again, until the explorer gave up the goal. The reason was

the repulsion of both sides of the passage, which was much stronger than the attraction of the waypoint and turned the robot away. We rebuilt the arena in the simulation and made several changes: the robot now moves straight through the passage if the way is free, the explorer does not turn around inside the passage, and the path stays in the middle of the passage. During these tests, we also found that the obstacle memory of the navigator almost never worked, because the laser scan was a few milliseconds newer than the odometry. After fixing this, the robot no longer touched the walls of the passage when rotating. However, the robot still gets stuck if it enters the passage at an angle, as the passage is then only a few centimetres wider than the robot. We estimate that passages of at least 0.9m work. These changes have not been tested on the real robot yet.

D. Time Management

Our work depended on parts of the project that were planned to be done by one team member. As these parts were not done, the whole project was delayed, and most of the work had to be done shortly before the deadline. This was one reason why we relied on an AI assistant (see Appendix). Apart from that, we also used it because it is a helpful tool for programming and debugging.

V. CONCLUSIONS

We implemented path and motion planning, localisation and exploration for the Robile and combined them in one ROS 2 package. In the simulation, all three tasks worked well: the robot reached its goals with an error of about 0.1m, the particle filter found the pose of the robot without a start pose, and the explorer created a complete map of the environment in about two minutes.

The step from the simulation to the real robot was harder than expected. Most problems were not in the algorithms themselves, but in the details: frame names, clock times, invalid sensor data, the real size of the robot and the network setup. The main lessons we learned are:

- The real size and shape of the robot matter. A robot that is modelled as a point touches walls with its corners.
- A simulation environment should be as similar as possible to the real test environment. Rebuilding our arena in the simulation helped us to find the causes of the problems with the narrow passage.
- Logs and visualisation (RViz) are essential to find the causes of problems, especially on the real robot.
- Simple potential fields have limits in narrow passages. A robot that aligns itself with the passage before entering it, or a local planner that considers the shape of the robot, would be the next improvement.

STATEMENT OF ORIGINALITY

I, the undersigned below, declare that this work has not previously been submitted to this or any other university and that it is, unless otherwise stated, entirely my own work. The report was, in part, written with the help of the AI assistants

Claude Code and ChatGPT as described in the appendix. I am aware that content generated by AI systems is no substitute for careful scientific work, which is why all AI-generated content has been critically reviewed by me, and I take full responsibility for it.

Date	Signature
Signature	Signature

APPENDIX

Use of AI Assistants

We used AI assistants during the whole project: mainly Claude Code (Anthropic), and in addition ChatGPT (OpenAI) for help with coding and debugging in task 1. Claude Code was used for the following parts:

- **Code:** Claude Code wrote the particle filter (task 2), the explorer (task 3) and the launch files, based on the methods of the course. It also made most of the later fixes to the path planner and the potential field navigator (task 1), which were written by our team.
- **Debugging:** We gave Claude Code the logs of the simulation and of the real robot, and it analysed them and suggested fixes, e.g. for the clock difference, the corner collisions and the narrow passage.
- **Simulation and tests:** Claude Code set up the Docker simulation, wrote test scripts that measured e.g. the explored area and the localisation error, and ran the tests in the simulation.
- **Git:** It created branches, commits and pull request descriptions. All commits made with its help are marked with “Co-Authored-By: Claude”.
- **Documentation:** It wrote parts of the README and a first version of this report, based on our notes in the report, the README, the code and our previous chats. We reviewed and adapted the text.

Typical prompts were short descriptions of the task or the problem, e.g. “the robot does not get through the narrow passage, it turns left and right in front of it” together with the logs of the robot. All results were checked by us in the simulation or on the real robot.